

CLASS PROJECT REPORT

Coverage-Based Testing of a *Java* Student Management System

LOGIC COVERAGE

INPUT SPACE PARTITIONING

GRAPH COVERAGE

JUNIT 5

PREPARED BY

Enas Hamed Alqarni — 2314104

Joury Ghorab — 2312080

Jana Akkad — 2313200

INSTRUCTOR

Dr. Mona Altherwi

Department of Software Engineering

Agenda

- | | | | |
|----|---------------------------------------|----|-----------------------------------|
| 01 | Software under test | 07 | ISP — ECC & BCC results |
| 02 | Why these three functions | 08 | Graph Coverage — CFG |
| 03 | Testing approach & criteria | 09 | Graph Coverage — NC & PPC results |
| 04 | Logic Coverage — <i>displayTop5()</i> | 10 | JUnit 5 tooling |
| 05 | Logic Coverage — PC & CACC results | 11 | Results summary |
| 06 | Input Space Partitioning — IDM | 12 | Conclusion |

Student Management System

A Java console application for managing a roster of students — add, search, update, delete, sort, filter, CSV/TXT import-export, merge two systems, and summary reports. Each record holds a name, ID, GPA, year, and department.

Chosen because it is fully backend, uses only the Java standard library, and contains enough decisions, loops, and switches to apply all three coverage models.

TOTAL LOC

≈ 967

Across six classes

CLASSES

6

3 functions tested

CLASS SIZES

CLASS	LOC	METHODS	TESTED
App.java	260	1	indirectly
Student.java	20	2	no
StudentSystem.java	327	13	yes — 3 fns
InputValidator.java	137	9	indirectly
CsvFileHandler.java	93	3	no
TxtFileHandler.java	130	1	no
TOTAL	≈ 967	29	—

Repository · github.com/Mohammed-3tef/Student_Management_System

Three functions, three structures

Each function exposes a different testing surface — a Boolean predicate, an input domain, and a control-flow graph — so the same codebase carries all three coverage models without overlap.

FUNCTION 1

`displayTop5()`

A conjunctive predicate guards every print: $\mathbf{C1} \wedge \mathbf{C2} \wedge \mathbf{C3}$. Three independent clauses — year match, slot-count, passing GPA — make this the natural target for clause-level logic coverage.

LOGIC COVERAGE

FUNCTION 2

`mergeStudentSystem(...)`

Output depends on the receiver's state and the overlap with the parameter, not just the input — the perfect candidate for partitioning the input domain into characteristics and blocks.

INPUT SPACE PARTITIONING

FUNCTION 3

`removeStudentByID(int)`

A for-loop with an early *return* and a fall-through not-found branch gives a non-trivial CFG with a loop-back edge — the right shape for Prime Path Coverage.

GRAPH COVERAGE

Three models, two criteria each

For every model we pair one weaker criterion with one stronger criterion that extends it — showing both ends of the cost–rigour scale. Five tests per criterion, thirty in total.

#	MODEL	FUNCTION UNDER TEST	CRITERION 1	CRITERION 2
1	Logic Coverage	displayTop5()	Predicate Coverage (PC)	Correlated Active Clause (CACC)
2	Input Space Partitioning	mergeStudentSystem(...)	Each Choice (ECC)	Base Choice (BCC)
3	Graph Coverage	removeStudentByID(int)	Node Coverage (NC)	Prime Path Coverage (PPC)

TESTING TOOL

JUnit 5 (Jupiter)
@Test · @BeforeEach · @DisplayName · assertions
library · stdout capture.

TEST BUDGET

30 tests
5 per criterion × 2 criteria × 3 models.

displayTop5() — one predicate, three clauses

PREDICATE UNDER TEST

$$P = C1 \wedge C2 \wedge C3$$

C1 **Year match** — student.year equals the section being printed.

C2 **Slot open** — count ≤ 5, the top-5 list is not yet full.

C3 **Academically passing** — student.GPA ≥ 2.0.

TRUTH TABLE — ALL 2³ ROWS

#	C1	C2	C3	P	USED BY
1	T	T	T	T	PC-01 / CACC anchor
2	F	T	T	F	PC-02 · C1 active
3	T	F	T	F	PC-03 · C2 active
4	T	T	F	F	PC-04 · C3 active
5	F	T	F	F	PC-05 · two false
6	F	F	T	F	subsumed
7	T	F	F	F	subsumed
8	F	F	F	F	subsumed

PC & CACC — clause values per test

PREDICATE COVERAGE (PC) · P MUST TAKE T AND F

TEST ID	C1	C2	C3	P	NOTE
TC-LC-PC-01	T	T	T	T	All clauses true
TC-LC-PC-02	F	T	T	F	C1 flipped
TC-LC-PC-03	T	F	T	F	C2 flipped
TC-LC-PC-04	T	T	F	F	C3 flipped
TC-LC-PC-05	F	T	F	F	Two clauses false

P reaches both T (row 1) and F (rows 2–5) → PC satisfied.

CORRELATED ACTIVE CLAUSE (CACC) · EACH C_i INDEPENDENTLY FLIPS P

TEST ID	C1	C2	C3	P	ACTIVE
TC-LC-CACC-01	T	T	T	T	Anchor
TC-LC-CACC-02	F	T	T	F	C1
TC-LC-CACC-03	T	F	T	F	C2
TC-LC-CACC-04	T	T	F	F	C3
TC-LC-CACC-05	F	T	F	F	Two false

Rows CACC-02..04 prove each clause alone can flip P → CACC satisfied.

mergeStudentSystem — four characteristics

The merge outcome depends on the receiver's state and the overlap with the parameter, not the call alone. Four characteristics partition the input domain; each is split into disjoint blocks.

CHAR.	DESCRIPTION	BLOCKS
C1	Size of the receiving (current) system	B1 empty · B2 one · B3 many
C2	Size of the incoming (other) system	B1 empty · B2 one · B3 many
C3	ID overlap between the two systems	B1 none · B2 some · B3 all
C4	Name overlap between the two systems	B1 none · B2 some

IDM GRID	B1	B2	B3
C1	empty	one	many
C2	empty	one	many
C3	none	some	all
C4	none	some	—

Total: 3 × 3 × 3 × 2 = 54 cells; ECC needs ≥ 3, BCC needs 5.

ECC & BCC — block tuples per test

EACH CHOICE (ECC) · EVERY BLOCK APPEARS ONCE

TEST ID	C1	C2	C3	C4	TUPLE SUMMARY
TC-ISP-ECC-01	B2	B3	B1	B1	1 main + 3 fresh, no overlap
TC-ISP-ECC-02	B2	B1	—	—	1 main, incoming empty
TC-ISP-ECC-03	B1	B2	B1	B1	main empty, 1 fresh
TC-ISP-ECC-04	B3	B3	B3	B2	all IDs & names overlap
TC-ISP-ECC-05	B2	B3	B2	B1	one ID collision

All blocks of C1, C2, C3, C4 represented → ECC satisfied.

BASE CHOICE (BCC) · VARY ONE CHARACTERISTIC AT A TIME

TEST ID	C1	C2	C3	C4	VARIATION
TC-ISP-BCC-01	B3	B3	B1	B1	Base — all base blocks
TC-ISP-BCC-02	B1	B3	B1	B1	vary C1 (main → empty)
TC-ISP-BCC-03	B3	B1	B1	B1	vary C2 (incoming → empty)
TC-ISP-BCC-04	B3	B3	B3	B1	vary C3 (IDs → all overlap)
TC-ISP-BCC-05	B3	B3	B1	B2	vary C4 (names → some)

Base tuple **(B3, B3, B1, B1)** = typical merge: populated sides, no overlap.

removeStudentByID — six-node CFG

```
public void removeStudentByID(int ID) {  
    for (int i = 0; i < studentList.size(); i++) {  
        if (studentList.get(i).ID == ID) {  
            studentList.remove(i);  
            System.out.println("Student removed successfully!");  
            return;  
        }  
    }  
    System.out.println("Student not found!");  
}
```

NODES

N = {1, 2, 3, 4, 5, 6}

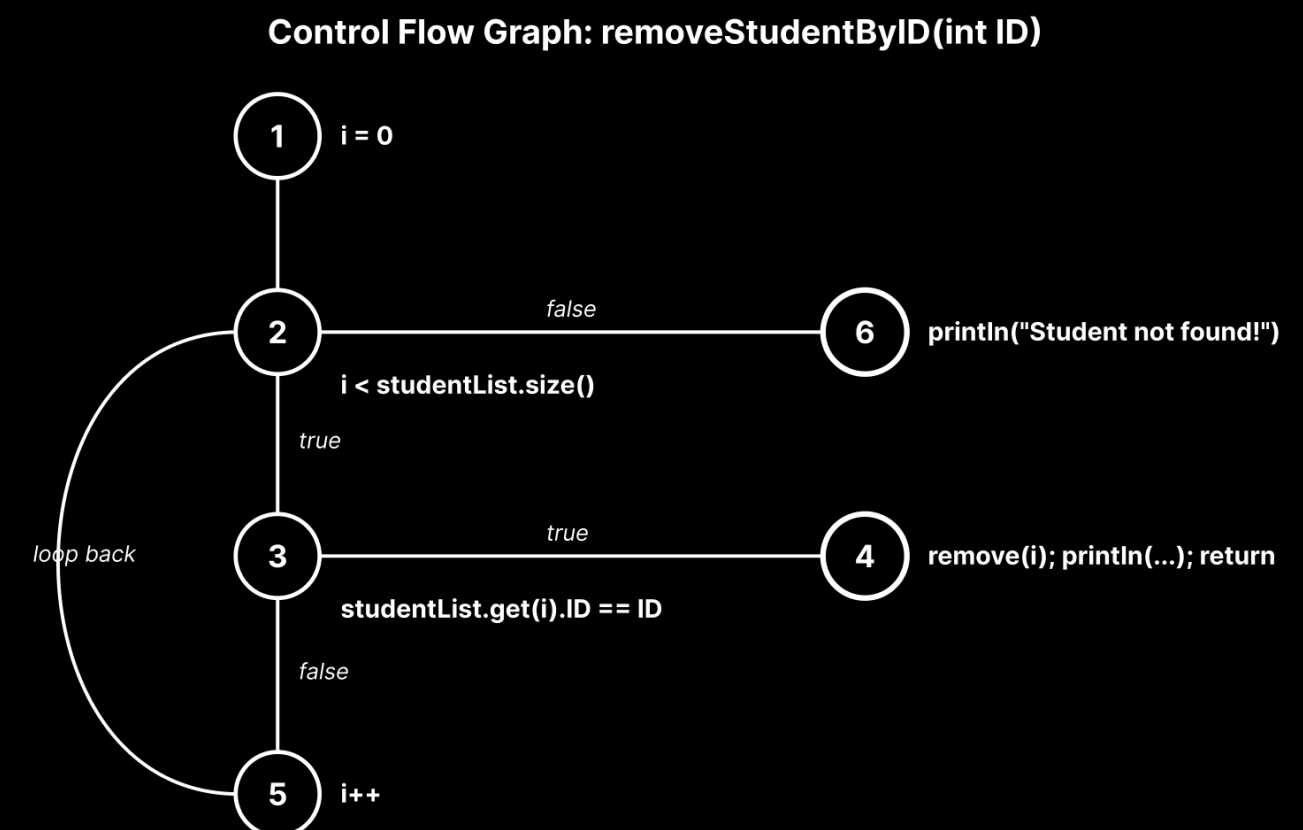
Initial: 1 · Final: {4, 6}

EDGES

(1,2) (2,3) (2,6)

(3,4) (3,5) (5,2)

CONTROL FLOW GRAPH



NC paths & the eight prime paths

NODE COVERAGE (NC) · PATH PER TEST

TEST ID	PATH WALKED	NODES COVERED
TC-GC-NC-01	1 → 2 → 6	{1, 2, 6}
TC-GC-NC-02	1 → 2 → 3 → 5 → 2 → 3 → 5 → 2 → 3 → 4	{1, 2, 3, 4, 5}
TC-GC-NC-03	1 → 2 → 3 → 5 → ... → 3 → 4	{1, 2, 3, 4, 5}
TC-GC-NC-04	1 → 2 → 3 → 5 → ... → 2 → 6	{1, 2, 3, 5, 6}
TC-GC-NC-05	1 → 2 → (3 → 5 → 2) × 4 → 3 → 4	{1, 2, 3, 4, 5}

Union of visited nodes = {1, 2, 3, 4, 5, 6} → NC satisfied.

PRIME PATH COVERAGE · EIGHT PRIME PATHS

PP	PRIME PATH	MEANING
PP1	[1, 2, 6]	loop body skipped (empty)
PP2	[1, 2, 3, 4]	match on first iter
PP3	[1, 2, 3, 5]	one mismatch
PP4	[2, 3, 5, 2]	loop-body cycle
PP5	[3, 5, 2, 3]	cycle (alt anchor)
PP6	[3, 5, 2, 6]	mismatch then exit
PP7	[5, 2, 3, 4]	increment then match
PP8	[5, 2, 3, 5]	increment, mismatch

Five test paths tour all eight prime paths → PPC satisfied.

JUnit 5 — features & example

Each Section 4 test case became a Java method. Per-test isolation via `@BeforeEach`; console output captured into a `ByteArrayOutputStream` so assertions can read what *StudentSystem* prints.

FEATURES USED

- `@Test`, `@BeforeEach`, `@DisplayName`
- Assertions — `assertEquals`, `assertTrue`, `assertFalse`, `assertNotNull`
- stdout capture via `System.setOut`
- Fresh *StudentSystem* per test — no state leaks

Three test classes — **LogicCoverageTest**, **InputSpacePartitioningTest**, **GraphCoverageTest** — ten methods each.

EXAMPLE — TC-GC-PPC-04

```
// PP3 + PP4 + PP5 + PP7
// one mismatch, then match on the 2nd iteration
@Test
@DisplayName("TC-GC-PPC-04  second-iteration match")
public void ppc_PP4_secondIterationMatches() {
    system.addStudent("Iman", 800, 3.0, "First", "CS", true); // idx 0
    system.addStudent("Jude", 801, 3.5, "Second", "IS", true); // idx 1

    system.removeStudentByID(801);
    releaseStdout();

    assertEquals(1, system.getStudentList().size());
    assertTrue (containsId(800));
    assertFalse(containsId(801));
    assertTrue (captured.toString().contains("removed successfully"));
}
```

Asserts the list state and the captured stdout — both verified in one test.

Thirty tests, all passing

PASS RATE

100%

30 of 30 tests passed.

FAILURES

0

CRITERIA

6

2 per model × 3 models

PASS / FAIL BY CRITERION

MODEL	CRITERION	TOTAL	PASSED	FAILED	PASS %
Logic	Predicate Coverage (PC)	5	5	0	100%
Logic	Correlated Active Clause (CACC)	5	5	0	100%
ISP	Each Choice (ECC)	5	5	0	100%
ISP	Base Choice (BCC)	5	5	0	100%
Graph	Node Coverage (NC)	5	5	0	100%
Graph	Prime Path Coverage (PPC)	5	5	0	100%
TOTAL	—	30	30	0	100%

What we learned

Three coverage models on the same project produced three very different styles of test — clause-flipping, block-tuple sampling, and CFG walking — and together they generated thirty unique cases, all passing on the current implementation.

RECOMMENDATIONS

- Return result objects or throw exceptions instead of printing to `System.out`.
- Override `equals` & `hashCode` on *Student*.
- Wire tests into a Maven or Gradle build and run on CI.

IN CLOSING

Thank *you.*

Enas Hamed Alqarni • Joury Ghorab • Jana Akkad
CCSW 323 • Dr. Mona Altherwi • University of Jeddah